HARVARD UNIVERSITY · PYTORCH INDIA · IIT BOMBAY

Don’t import it. Build it.

Vijay Janapa Reddi

Harvard University · ETH Zürich (sabbatical)

The xv6 of Machine Learning Systems.

An educational reference runtime: understand how PyTorch,

TensorFlow and JAX really work.

tinytorch.ai · mlsysbook.ai

BEFORE ANYTHING ELSE

Hands up if you have written this line.

```
loss.backward()
```

Most of this room. Good. That is the right answer.

NOW THE SECOND QUESTION

Keep it up if you can tell me what that line allocated.

How much memory. In what order. Freed when.

The distance between those two questions is this talk.

THE HISTORICAL PARALLEL

Machine learning education faces an operating systems crisis.

YEAR 2000 · OPERATING SYSTEMS

Linux had grown to millions of lines of C. Device drivers, architecture macros, backwards compatibility.

Students could not learn virtual memory because the page table walk was buried in 10,000 lines of x86 assembly.

TODAY · MACHINE LEARNING SYSTEMS

PyTorch has grown to millions of lines of C++, CUDA, template metaprogramming, and dispatch tables.

Students treat the framework as an oracle. When memory bandwidth saturates or loss explodes, they are left guessing.

Harvard's TinyTorch is to Machine Learning Systems as MIT's xv6 is to Operating Systems: a clean, self-contained reference framework you can read, build, and optimize in one semester.

WHY ANY OF THIS EXISTS

The world is rushing to build AI systems.

It is not engineering them.

That gap is what we mean by AI engineering.

WHO THE INDUSTRY IS ACTUALLY SHORT OF

Not the people who invent new architectures.

IN GOOD SUPPLY

Researchers proposing new models. Engineers who can call an API and fine-tune a checkpoint.

THE BOTTLENECK

The people who make existing architectures computationally viable. Who know when mixed precision holds, what checkpointing costs, why synchronization eats the speedup.

You cannot build that mental model out of a high-level API.
It only comes from building the thing.

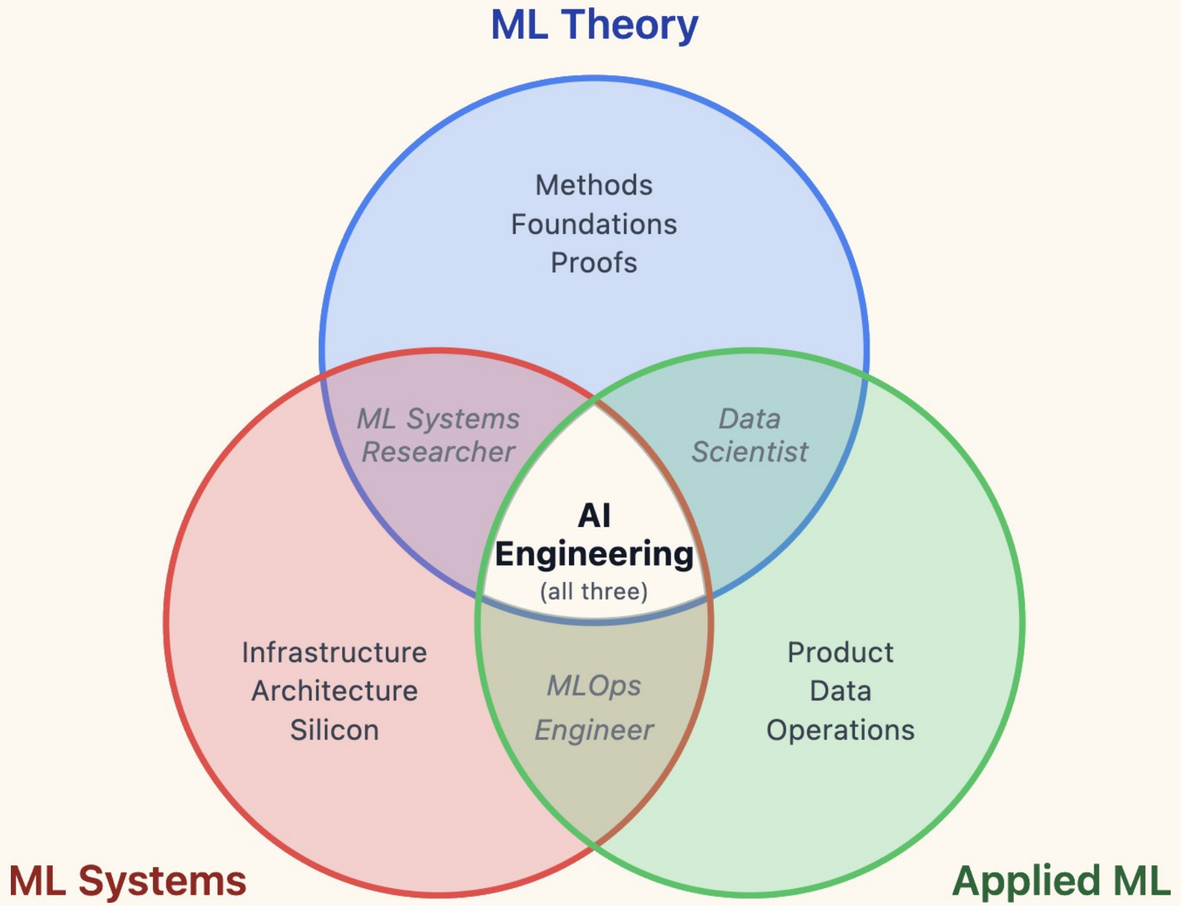
THE DISCIPLINE THIS NEEDS

AI Engineering sits where three fields overlap.

Not a specialization of any one of them. It has its own failure modes, its own costs, and its own body of knowledge.

It connects theory, systems and applications. It emphasizes the real trade-offs — efficiency, reliability, scale. And it is what moves an engineer from demo to production.

It does not have its curriculum yet.



AND IT SPANS THE WHOLE STACK

PyTorch is one layer of it.

Applications	end-use products, responsible AI
Operations	MLOps, monitoring, CI/CD
Serving	inference runtimes, deployment
Training	training loops, distributed training
Models	neural network architectures
Frameworks	PyTorch, TensorFlow, compilers
Hardware	GPUs, TPUs, accelerators

you are here

Every layer is engineering.

Most curricula cover the middle two and stop. The rest is where systems actually fail, and where almost nobody is trained.

Machine learning systems is the whole stack. Today I have time for one layer of it.

LOOK AT THE REST OF TODAY'S PROGRAM

Every talk after this one is a fight the framework has with hardware.

ON THIS STAGE TODAY			YOU BUILD IT IN
10:20	**Digital CMOS accelerators**	Getting compute to the edge	17 Acceleration
11:00	**PyTorch Compile**	Graph capture and kernel fusion	06 Autograd 14 Profiling
11:35	**vLLM and llm-d**	KV cache, memory, scheduling	18 Memoization
12:25	**ExecuTorch**	Shrinking a model to fit the device	15 Quantization 16 Compression

You cannot follow any of them while backward() is still magic.

WE ALREADY KNOW HOW TO TEACH THIS

Computer engineering solved it sixty years ago.

HOW WE TEACH A PROCESSOR

transistors → logic gates → arithmetic units → a CPU

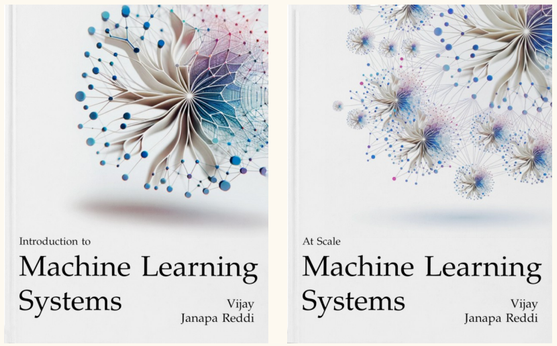
HOW TINYTORCH TEACHES A FRAMEWORK

tensors → operations → layers → an ML system

Each layer is a working system that makes the next one possible. Nobody teaches CPUs by handing you a CPU.

SO WE BUILT THE CURRICULUM FOR IT

Three materials. One discipline.

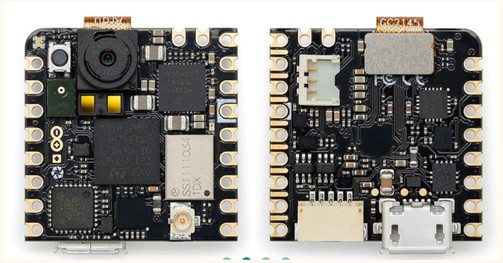


READ · mlsysbook.ai

The textbook

Four volumes, 33 chapters, free.

Theory and concepts across the whole ML systems stack.



Seeed · Arduino · Raspberry Pi

BUILD · mlsysbook.ai/kits

Hardware Kits

Affordable, vendor-neutral boards.

Hands-on labs for real-world deployment.

tiny TORCH

BUILD · tinytorch.ai

TinyTorch

Build the framework. Understand systems. Recreate history.

Debug anything.

tiny
TORCH

Don’t import it. Build it.

Twenty modules. Pure Python. Tensors through transformers,

on a laptop with four gigabytes of RAM and no GPU.

A HANDS-ON WAY TO BUILD A FRAMEWORK FROM SCRATCH

What TinyTorch actually is.

Build each piece

Tensors, autograd, attention.

No magic imports.

Recreate history

Perceptron → CNN →

Transformers → MLPerf.

Understand systems

Memory, compute,

optimization trade-offs.

Debug anything

OOM, NaN, slow training —

because you built it.

THE QUESTION EVERY STUDENT ASKS

Why build it when you could import it?

Why Build Instead of Use?

"Building systems creates irreversible understanding."

Traditional ML Education

```
import torch
model =
torch.nn.Linear(784, 10)
output = model(input)
# When this breaks, you're
stuck
```

Problem: You can't debug what you don't understand.

TinyTorch: Build → Use → Reflect

```
# BUILD it yourself
class Linear:
    def forward(self, x):
        return x @
self.weight + self.bias

# USE it on real data
loss.backward()  # YOUR
autograd
```

Advantage: You can debug it because you built it.

Building systems creates irreversible understanding.

Once you have implemented autograd you cannot unsee the computational graph.

Once you have profiled your own allocations you cannot unknow the cost.

That is the whole pedagogical bet.

THE BUILD ORDER

You cannot optimize a training loop you have not written yet.

Foundation 01 – 08

Tensors, activations, layers, losses, dataloader, autograd, optimizers, training

Architecture 09 – 13

Conv2d and CNNs, tokenization, embeddings, attention, transformer blocks

Optimization 14 – 19

Profiling, quantization, compression, acceleration, KV cache, benchmarking

Torch Olympics 20

The capstone. Your framework, measured against everyone else's

Each tier depends on the one below it. There is no skipping ahead.

HOW YOU KNOW YOUR IMPLEMENTATION IS CORRECT

Sixty-seven years of ML history, running on nothing but your code.

1958

Perceptron

The first trainable learning algorithm

1969

XOR crisis

Minsky and Papert's limit, broken

1986

MLP revival

Backpropagation enables deep learning

1998

CNN revolution

75% on CIFAR-10, or it does not count

2017

Transformers

Attention launches the LLM revolution

2018+

MLPerf

Production optimization, benchmarked

A milestone unlocks only when the modules beneath it work.
Your autograd is correct because your network learns.

LIVE EXECUTION PROOF · MILESTONE 03

It is not a simulation.
It is running your actual code.

1986 MLP Revival

Training a 2,410-parameter neural network on TinyDigits.

- No torch imports anywhere in the process.
- Strided tensor memory allocated by hand.
- Gradients routed through your autograd tape.
- 100% generalization accuracy on held-out digits.

When a student sees this screen, they know they understand deep learning.

DESIGN DECISION 01

Match the production API exactly.

TinyTorch

```
# tinytorch
for images, labels in train_loader:
    logits = model(images)
    loss = loss_fn(logits, labels)

    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
```

PyTorch

```
# pytorch
for images, labels in train_loader:
    logits = model(images)
    loss = loss_fn(logits, labels)

    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
```

Only the import differs. Nothing you learn here has to be unlearned later.

DESIGN DECISION 02

Systems from Module 01.

Module 01 ships memory_footprint() before it ships matrix multiplication.

```
x = Tensor.randn(32, 3, 224, 224)
x.memory_footprint()
# 19.3 MB   <- one batch
```

You compute it. You do not read it.

Adam costs roughly 3× the optimizer memory of SGD, because you allocated those buffers yourself and then measured them.

Memory stops being a thing that happens to you and starts being a thing you decide.

THE CORE FRAMEWORK INVARIANT · MODULES 01 & 06

Broadcasting is a zero stride forward, and a sum reduction backward.

FORWARD PASS · ZERO-STRIDE VIEW

```
x.shape = (4, 3)    b.shape = (3,)

x + b  # expands b to (4, 3)
```

- Stride along batch axis is set to 0: strides = (0, 1)
- Zero allocations. Shared physical buffer.
- Every row reads the exact same bytes in memory.

BACKWARD PASS · MULTIVARIABLE SUM

```
_reduce_broadcast_grad(grad, orig_shape)
```

- Multivariable chain rule: dL/db = sum(dL/dy_i)
- Must sum across every axis where stride was 0.
- Dual contract: forward stride 0 forces backward sum.

This single contract governs Linear bias, Convolutions, Embeddings, Attention masks, and LayerNorm. Miss it, and gradients silently corrupt.

DEEP DIVE · MODULE 06

The computational graph is just the parent pointers.

```
class Tensor:

    def __mul__(self, other):
        out = Tensor(self.data * other.data)
        out._parents = (self, other)      # the edge

        def _backward():                  # the local rule
            self.grad  += other.data * out.grad
            other.grad += self.data  * out.grad

        out._backward = _backward
        return out

    def backward(self):
        self.grad = 1.0
        for node in reversed(topo_sort(self)):
            node._backward()
```

Seventeen lines.

The graph is not a data structure somebody hands you. It is the parent pointers you just wrote.

And backward() is one reverse walk over them, calling a local rule at each node.

After this module you cannot look at PyTorch's autograd and see magic.

HARDWARE REALITY · MODULE 07

The 16-byte parameter law of modern training.

Weights

4 bytes

FP32 master parameters

Gradients

4 bytes

dL/dw from backward pass

Momentum (m)

4 bytes

First moment buffer

Variance (v)

4 bytes

Second moment buffer

A 7-Billion Parameter Model Requires 112 GB of VRAM Just to Sit in Memory.

7×10^9 params × 16 bytes = 112 GB before storing a SINGLE activation tensor.

Optimizer state, not the model, is why deep learning requires clusters and ZeRO memory sharding.

DEEP DIVE · MODULE 12

The last eight years of AI, in seven lines.

```
def attention(q, k, v, mask=None):
    d_k = q.shape[-1]
    scores = (q @ k.transpose(-2, -1)) / sqrt(d_k)

    if mask is not None:
        scores = scores.masked_fill(mask == 0, -inf)

    weights = softmax(scores, axis=-1)
    return weights @ v
```

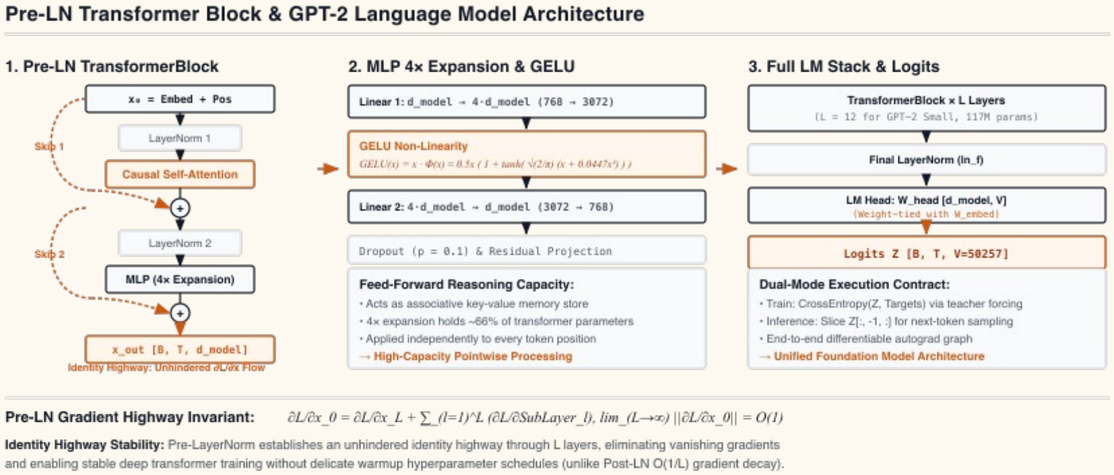
You write every one of them.

Not a call into somebody's kernel. The scaling, the mask, the softmax, the weighted sum.

Then Module 18 makes you cache the K and V. **That cache is the reason vLLM exists.**

DEEP DIVE · MODULE 13

The Transformer Block: Residual highways for gradients.



The Pre-LN Invariant

```
x_{l+1} = x_l + F(LayerNorm(x_l))
```

Sub-layers never replace the token's representation. They normalize a copy, compute an additive delta, and add it back to the stream.

In the backward pass, the gradient flows directly down the residual highway: dL/dx_l = dL/dx_{l+1} + dL/dF.

This simple addition is why 100-layer transformers train without vanishing gradients.

LIVE EXECUTION PROOF · MILESTONE 05

Attention is all you need. Running on your runtime.

2017 Transformer Era

Training a 103,709-parameter transformer from scratch.

- MultiHeadAttention & LayerNorm built by hand.
- Passes sequence reversal (96%) and copying (100%).
- Dynamic prompt conditioning ([R] vs [C]) at 99.7% accuracy.
- Zero torch imports — your tensors, your autograd tape.

The foundation of GPT and modern LLMs, executing entirely on code the student wrote.

THE ACCIDENT THAT TURNED OUT TO BE THE FEATURE

Your first Conv2d is going to be catastrophically slow.

YOUR CONV2D

97 seconds

PYTORCH, SAME BATCH

10 milliseconds

The argument for vectorization stops being something they read and starts being something that happened to them.

The Roofline Model: Where does the time actually go?

ARITHMETIC INTENSITY (I)

`I = FLOPs / Memory Bytes Loaded`

• Compute-Bound: I > Ridge Point (e.g., Matrix Mul prefill). Running at peak processor throughput.

• Memory-Bound: I < Ridge Point. Running at memory bandwidth wall.

THE AUTOREGRESSIVE REALITY

`Token Generation: I ≈ 0.5 FLOPs/Byte`

• For batch size 1 decode, we stream every parameter from memory just to multiply it by 1 float!

• 99% of execution time is spent waiting on DRAM traffic, NOT computing.

Every acceleration in Part III is a way to lower memory bytes Q or raise arithmetic intensity I. Without this mental model, optimization is blind guessing.

DEEP DIVE · THE OPTIMIZATION TIER, TWO AXES

Six modules. Three orders of magnitude.

BASELINE

Your model

100 MB

0.5 tok/s

the honest starting point

→

SHRINK · 15

Quantization

25 MB

4× smaller

INT8 weights, same accuracy

→

SHRINK · 16

Compression

12 MB

8× smaller

pruned, 94% accuracy

→

SPEED · 18

KV cache

50 tok/s

100× faster

stop recomputing the past

→

SPEED · 17

Acceleration

500 tok/s

1000× total

batch and vectorize

→

PROVE · 19

MLPerf

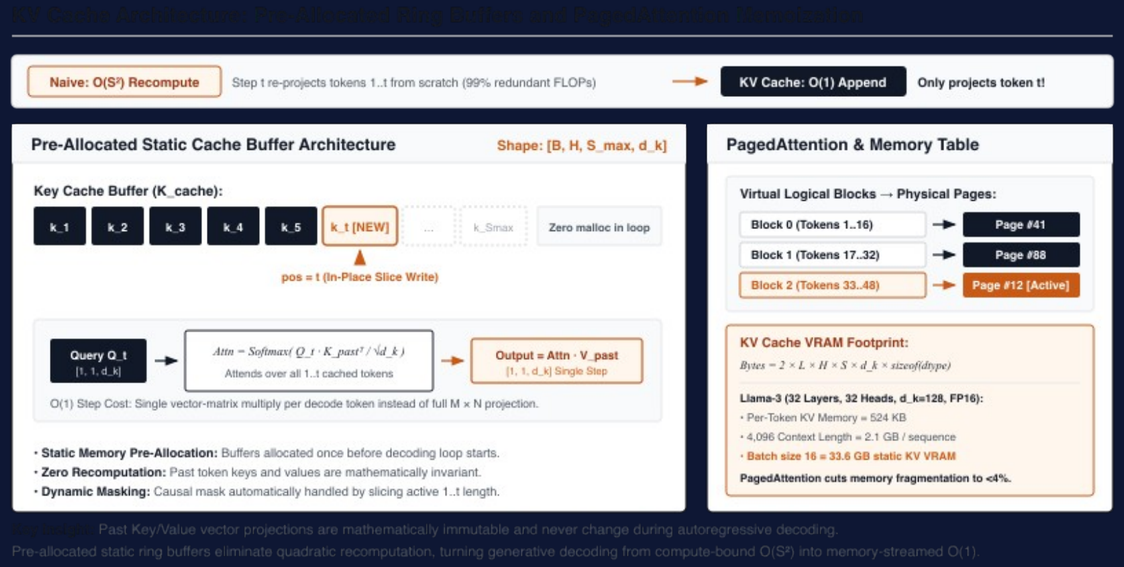
measured

and defensible

benchmark it like production

This is what ExecuTorch and vLLM do for a living.
You will have done it by hand first.

Why vLLM exists: Eliminating O(T²) recomputation.



The Generative Dilemma

Without cache: Generating token T requires computing attention over all 1, 2, ..., T-1 tokens again. Total cost = O(T²).

With KV Cache: Past keys and values are invariant under causal masking. Store them in an append-only ring buffer.

Per-token decode collapses from O(T) matrix multiplications to a single vector-matrix multiply (O(1)).

vLLM's PagedAttention is operating systems virtual memory applied to this exact tensor.

THIS IS NOT A PILE OF EXERCISES

Forty competencies. Thirty-six covered.

Table 2. ML Systems Competency Matrix (Single Node). The 40 cells (8 knowledge areas × 5 capability levels) characterize the foundational competencies that AI engineers require. Rows progress from foundational concepts (tensors, operations) through learning mechanics (graphs, optimization) to production concerns (efficiency, deployment). Columns progress from conceptual understanding (Knows) through quantitative reasoning (Measures) and implementation (Implements) to diagnostic skill (Debugs) and performance engineering (Optimizes). Distributed systems concepts (gradient synchronization, model parallelism, communication overhead) represent a natural subsequent learning objective building on these foundations. This matrix guided TinyTorch's curriculum design. Section 6.3 maps the 20 modules to this framework.

Knowledge Area	Knows	Measures	Implements	Debugs	Optimizes
Tensors & Memory	Layouts, strides, dtypes	Bytes, peak allocation	Tensor class, broadcasting	OOM, memory leaks	In-place ops, pooling
Operations & Compute	Broadcast rules, semantics	FLOPs, arithmetic intensity	MatMul, Conv2d, activations	Shape mismatch, instability	Vectorization, fusion
Graphs & Differentiation	Computation graphs, memory lifecycle	Gradient memory, node count	Autograd, backward()	Vanishing/ exploding grads	Checkpointing
Optimization & Training	Optimizer state, update costs	State size, convergence rate	Optimizers, training loop	Non-convergence, NaN	LR scheduling, clipping
Architectures & Models	Parameter scaling, compute patterns	Parameters, receptive field	Layers, blocks, full models	Accuracy plateau, overfit	Pruning, distillation
Pipelines & Data	Batching, shuffling, prefetch	Throughput, CPU utilization	Dataset, DataLoader	Data bottleneck, stalls	Parallel loading, caching
Efficiency & Compression	Quantization, pruning tradeoffs	Compression ratio, accuracy delta	INT8, magnitude pruning	Accuracy degradation	Calibration, sparsity
Deployment & Serving	Batch vs streaming, caching	Latency p50/p99, throughput	KV cache, model export	Timeout, memory bloat	Dynamic batching

You do not need to read this.

Eight knowledge areas by five capability levels — Knows, Measures, Implements, Debugs, Optimizes.

The curriculum was designed against it, not assembled from whatever was easy to teach.

36 full coverage

3 partial

1 deliberate gap

The one gap is parallel data loading. It breaks the four-gigabyte floor, so we left it out on purpose.

THE DECISION WE ARE LEAST WILLING TO TRADE

Four gigabytes of RAM. No GPU. No cloud account.

We decided the hardware floor before we decided the feature list.

That cost us GPU support. It bought us five-year-old laptops, shared computer labs, and universities whose network policy would have blocked anything else.

What we gave up

GPU kernels. Distributed training. Anything that needs more than one machine.

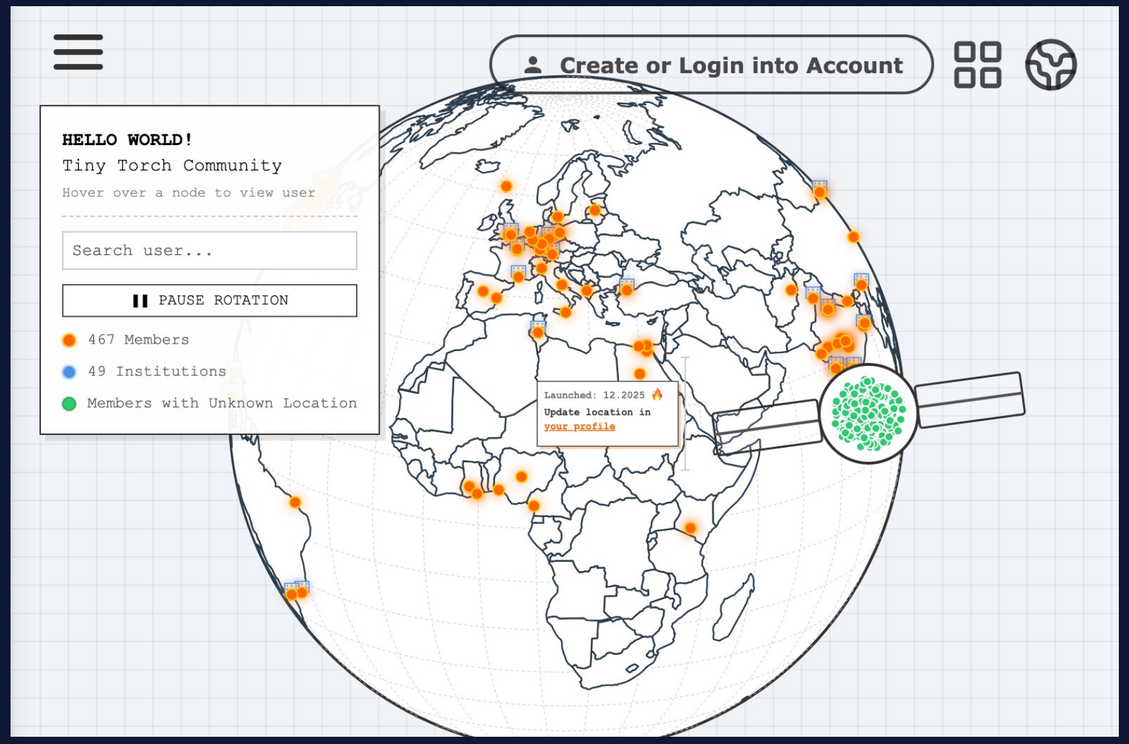
Work out who you are willing to exclude before you start.

Adoption is heaviest exactly where GPU access is hardest.

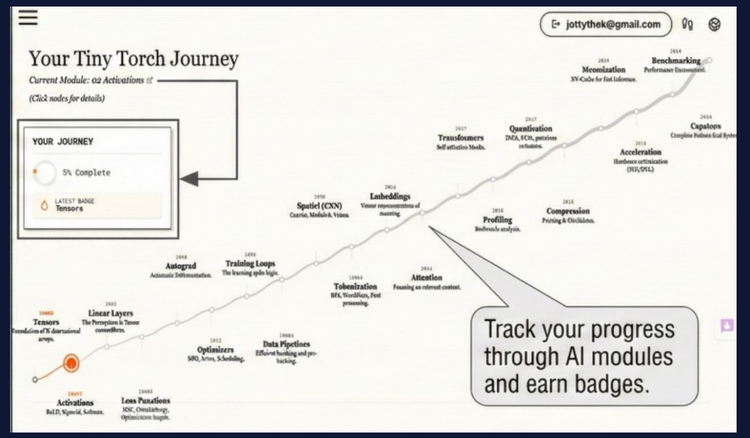
That is not a side effect. That was the design target.

LAUNCHED DECEMBER 2025

A global network of learners.



Every orange dot is a person who started Module 01.



Track your progress through all twenty modules and earn badges.

28,000+ GitHub stars

120+ contributors

92 institutions

209 countries

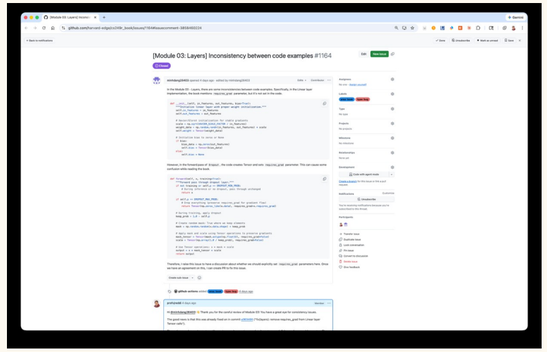
AND IT IS NOT A DEMO

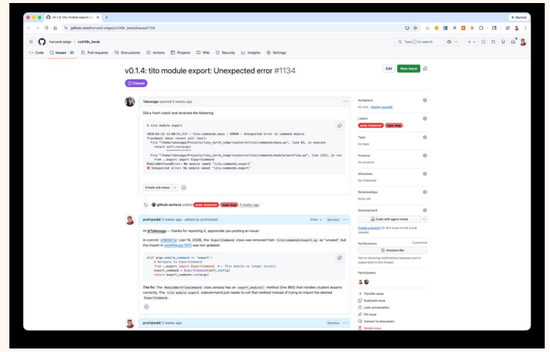
People are finding our bugs.

🔥 TinyTorch Contributors

Vijay Janapa Reddi	kai	Dang Truong	Didier Durand	Karthik Dani	Avik De	Takosaga	rnjema	joeswagson
AndreaMattiaGaravagno	Amir Alasady	jettythek	wzz	Ng Bo Lin	keo-dara	Wayne Norman	Ilham Rafiqin	Oscar Flores

Ninety-five-plus contributors, most of whom we have never met.





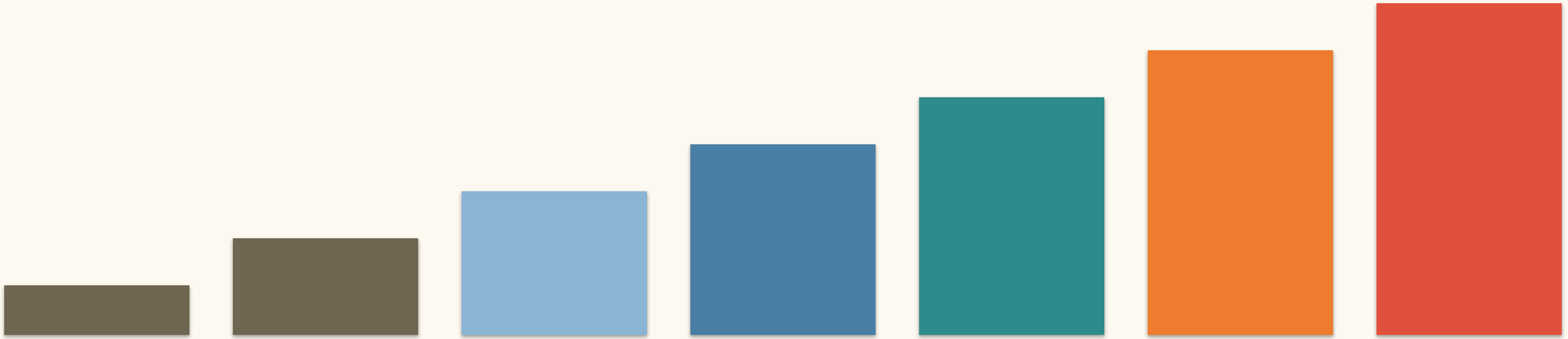
"404 error when installing TinyTorch." "Typo on Xavier/Glorot definitions throughout module."

A curriculum with an issue tracker is a curriculum that can be wrong in public.
That is the only kind worth adopting.

AN HONEST SLIDE ABOUT GROWTH

Each star is one more person who thinks AI systems should be engineered.

Today we are here → **28,000 stars** · 358,000 readers · 209 countries



1
learners

10
learners

100
learners

1,000
learners

10,000
learners

100,000
learners

1M
learners

The goal is not the number. The goal is the training capacity to reach a million AI engineers by 2030.

THE PART I DID NOT BUILD

Some of the people who built this are in this room.

10:40 · Student Showcase

TinyTorch contributors, on this stage,

in twenty minutes.

Ninety seconds of my slot is worth less than ten of theirs. Please stay for it.

ZOOM ALL THE WAY OUT

This is a discipline, not a specialization.

Computer Engineering

How machines compute

Settled curriculum

Seventy years old

Software Engineering

How systems get built and stay alive

Settled curriculum

Fifty years old

AI Engineering

How learning systems get built, deployed and kept alive

No settled curriculum

And it is the one hiring

The third column is not a branch of the first two. It has its own failure modes, its own costs, and its own body of knowledge.

THE PART I MOST WANT YOU TO HEAR

You have been told AI is coming for your job.

WHAT IS BEING AUTOMATED

Writing the glue. Calling the API. Reproducing the tutorial. The work that sits on top of the abstraction.

WHAT IS MULTIPLYING

Deciding what runs where, on which hardware, at what cost, with which failure modes. The work underneath it.

Every model that gets deployed needs somebody who understands the system underneath it.

That number is going up. Almost nobody is being trained for it.

That is not a threat. That is an opening.

THE HONEST SLIDE

Four things we cannot claim.

No learning-outcome data

The design leans on fifty years of evidence that build-it-yourself works. That is a strong prior. It is not a result.

Single node, CPU only

It teaches memory and compute well. It teaches nothing about GPU kernels.

Nothing distributed

No gradient synchronization, no sharding, no multi-node anything.

Parallel data loading is empty

We mapped the competency and deliberately left it out. It breaks the 4 GB floor.

Those are not roadmap items. They are open positions, and IIT Bombay could fill them.

FIVE THINGS, IN ORDER OF HOW MUCH EACH MOVES

Where you come in.

01	**Try it**	One line, runs locally, no account. Do it during the coffee break.
02	**Review a module**	If you work on PyTorch internals, an hour on our autograd is worth an enormous amount. Whatever we teach becomes what a cohort believes.
03	**Pilot a tier**	Foundation fits an undergraduate systems module. All twenty fit a semester. We want the failure reports more than the success stories.
04	**Write what we cannot**	Distributed training. GPU acceleration. Parallel data loading.
05	**Adopt it and say so**	It tells the next department this is a real option and not an experiment.

```
curl -sSL mlsysbook.ai/tinytorch/install.sh | bash
```

tiny TORCH

Don’t import it. Build it.

There are a million people who can call an API.

There are very few who can open it when it breaks.

Go and be one of them.

Vijay Janapa Reddi

Harvard University · ETH Zürich (sabbatical)

THE REFERENCE RUNTIME & COURSE

https://tinytorch.ai

Build PyTorch from scratch in Python: 20 hands-on modules covering tensors, autograd, convolution, transformers, and optimizers.

Interactive browser runtime with automated test suites.

SCAN TO RUN

THE OPEN TEXTBOOK

https://mlsysbook.ai

Machine Learning Systems: Principles and Practice.

A comprehensive curriculum bridging deep learning algorithms, systems architecture, distributed training, hardware, and serving.

SCAN TO READ